

Machine Unlearning on Trajectory Data: An Experimental Analysis

Yuanjun Liu
School of Computer
Science and Technology
Soochow University
Suzhou, Jiangsu, China
School of Data Science
Lingnan University
Hong Kong, China
yjliu1@stu.suda.edu.cn

Jiaxing Shen
School of Data Science
Lingnan University
Hong Kong, China
jiaxingshen@ln.edu.hk

Weiqi Wang
Xi'an Jiaotong University
Xi'an, Shaanxi, China
weiqi.wang@xjtu.edu.cn

Jingwen Li
Sichuan Normal University
Chendu, Sichuan, China
lijingwen@sicnu.edu.cn

Jiajie Xu
School of Computer
Science and Technology
Soochow University
Suzhou, Jiangsu, China
xujj@suda.edu.cn

Lei Zhao
School of Computer
Science and Technology
Soochow University
Suzhou, Jiangsu, China
zhaol@suda.edu.cn

Haoran Xie
School of Data Science
Lingnan University
Hong Kong, China
hrxie2@gmail.com

An Liu*

School of Computer
Science and Technology
Soochow University
Suzhou, Jiangsu, China
anliu@suda.edu.cn

Abstract

Machine Learning (ML) models have become essential for trajectory data analysis, supporting tasks such as similarity learning, map matching, simplification, and recovery. While learning from trajectory data enables personalized and location-based services, the ability to unlearn specific samples is equally crucial for removing outdated information, mitigating privacy risks, and complying with emerging data regulations. Existing machine unlearning methods, however, have been developed almost exclusively for the Computer Vision (CV) domain and their applicability to trajectory learning remains largely unexplored. In this work, we present the first systematic study of machine unlearning for trajectory data. We investigate (i) whether and how representative unlearning algorithms can be adapted to trajectory learning models, (ii) how to evaluate their effectiveness, and (iii) how unlearning outcomes vary across datasets, tasks, and unlearning scenarios. To this end, we design a comprehensive experimental framework covering ten unlearning algorithms, four key trajectory learning tasks, and multiple evaluation metrics on three real-world datasets. Our study reveals the unique characteristics and sensitivities of unlearning on trajectory data, provides insights into the limitations of existing approaches, and highlights open challenges for future research. The results establish a foundation for building standardized benchmarks and advancing machine unlearning in trajectory analytics. The code is available at <https://github.com/yuanjun-liu/TrajMU>.

*An Liu is the corresponding author, and is also with the Institute of Applied Ethics, Soochow University.



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD 2026, Jeju Island, Republic of Korea.*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817450>

CCS Concepts

• Information systems → Location based services; • Security and privacy → Security services; • Computing methodologies → Machine learning.

Keywords

Trajectory Data Mining, Machine Unlearning, Trajectory Similarity, Trajectory Simplification, Map Matching, Trajectory Recovery

ACM Reference Format:

Yuanjun Liu, Jiaxing Shen, Weiqi Wang, Jingwen Li, Jiajie Xu, Lei Zhao, Haoran Xie, and An Liu. 2026. Machine Unlearning on Trajectory Data: An Experimental Analysis. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026)*, August 9–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3770855.3817450>

1 Introduction

With the widespread availability of GPS devices and the increasing use of location-based services [8, 27, 48], massive volumes of trajectory data are being collected, typically represented as sequences of points containing location and timestamps. Consequently, various trajectory learning models have been proposed to support fundamental trajectory tasks, including the TMN [80] and BLUE [89] models for trajectory similarity learning, S3 [22] and MLSimp [69] models for trajectory simplification, GraphMM [49] and MMA [71] models for trajectory map matching, and TERI [12] and TRMMA [71] models for trajectory recovery, among others. For example, a collected trajectory is first matched to the map to reduce GPS noise during collection and to prepare for map-based services, then simplified to reduce storage space, and finally recovered to compute representations via similarity learning to serve k -NN queries.

While trajectory models are expected to learn information from data, it is equally important for them to *unlearn* information. On

the one hand, trajectory data, collected with spatial and temporal information, lose their utility if they become outdated or pass through a spatial area that has changed its function. On the other hand, trajectory learning models may memorize sensitive information of users [6, 71], raising ethical and security concerns. Thus, legal frameworks, such as the General Data Protection Regulation (GDPR) [20], have been enacted to guarantee users the right to request the removal of their data. Therefore, unlearning information from learned trajectory models is crucial for both providing better services and protecting user privacy.

Machine unlearning, which aims to erase the influence of specified samples from trained ML models, can remove out-of-date data and protect the privacy of specific individuals. Given the original model θ trained on the original dataset D and the unlearning set $D_u \subset D$, the unlearned model θ_u is expected to be similar to the model retrained on the remaining set $D_r = D \setminus D_u$. Although retraining unlearns exactly, it is computationally prohibitive. For example, JD.com reportedly collects 1 TB of trajectory data every day [42], retraining models on such a volume of data is impractical. Consequently, numerous unlearning algorithms have been proposed to approximate retraining. These include statistic-based methods [5, 62], adversarial-based methods [16, 70], information matrix based methods [25, 26], teacher-student based methods [17, 38], and perturbation-based methods [24, 93], among others.

However, existing machine unlearning research mainly focuses on the CV field, the machine unlearning on trajectory data remains unexplored. The following questions need to be answered. First, it is unclear whether existing unlearning algorithms can be directly applied to trajectory learning models. If not, what are the reasons, and can we make adaptations? Second, the performance of unlearning algorithms on trajectory data is unknown. How should we evaluate the performance and unlearning efficacy on trajectory data? Do existing unlearning algorithms perform as well as they do in CV tasks, and are they consistent across different trajectory learning tasks and scenarios? Which algorithms should be used for different trajectory learning tasks and scenarios? Third, if the unlearning algorithms do not perform as expected, what are the reasons? The characteristics of trajectory data and different trajectory learning tasks need to be clarified. Do these characteristics affect unlearning and evaluation, and if so, how? Fourth, do existing unlearning algorithms satisfy the demands of unlearning trajectory data? What are the remaining challenges of unlearning on trajectory data?

To answer the above questions, we initiate the study of unlearning trajectory data in learned trajectory models in this paper. Specifically, we study its effect on four key trajectory tasks and inform the community of our findings and open problems regarding unlearning trajectory data. Section 2 discusses related works. Section 3 introduces the definitions of four trajectory learning tasks and the details of selected state-of-the-art (SOTA) trajectory learning models for each task. Section 4 provides details on ten machine unlearning algorithms, the adaptations required for specific algorithms, and the reasons why several unlearning algorithms cannot be applied to trajectory data directly. Moreover, it is challenging to measure how well a model has truly forgotten the deleted cohort of data after an unlearning algorithm is applied, as defining an appropriate set of evaluation metrics is an open problem in and of itself. We elaborate on these metrics in Section 5. Then, we conduct

comprehensive experiments on three real-world trajectory datasets, two unlearning scenarios, four trajectory learning tasks and models, and ten unlearning algorithms in Section 6. Experimental results demonstrate the performance of unlearning algorithms on different datasets, metrics, and scenarios. Section 7 discovers the characteristics of trajectory data and learning tasks, as well as their effects on unlearning algorithms. Section 8 further discusses several open problems of unlearning on trajectory data and suggests potential solutions. Finally, we conclude our work in Section 9.

The main contributions of this paper are as follows.

- We initiate the study of unlearning on trajectory data, investigating four trajectory learning tasks with corresponding models and ten unlearning algorithms, detailing how to combine the algorithms with trajectory learning models, and how to set appropriate metrics for measuring performance on both trajectory tasks and unlearning efficacy.
- We conduct comprehensive experiments with three real-world trajectory datasets and two unlearning scenarios, and analyze the results to answer a large number of key questions and summarize the learned lessons and open problems, offering valuable insights to guide future research.
- The open-source code ¹ unifies the development process of unlearning on trajectory data, with implemented unlearning algorithms serving as a unified benchmark.

2 Related Work

Our study connects trajectory learning and machine unlearning, so we introduce the literature on both of them.

2.1 Trajectory Data Mining

Trajectory Similarity Computation. Trajectory similarity computation methods can be divided into two groups, rule-based methods, e.g., DTW [1], STS [40], and ITS [50], and learning-based methods, such as BLUE [89], LH-plugin [68], and GPE [51]. Since rule-based methods rely on pre-defined rules, which cannot take advantage of the information inside the data, and are typically time-consuming, learning-based methods have become more popular recently. Learning-based approaches use neural networks, such as RNNs, GNNs, and Transformers, to convert trajectories into representation vectors, subsequently computing the similarity of vectors as the similarity of trajectories. t2vec [43], At2vec [47], NeuTraj [81], TMN [80], TrajCL [7], and RSTS [14] utilize RNNs or Transformers to represent trajectories. GTS [90], KGTS [15], GRLSTM [91], TrajGAT [82], ST2Vec [21], START [34], and JGRM [55] combine GNNs on road networks with RNNs or Transformers to represent trajectories. MMTEC [46] leverages the CDE network for trajectory representation. GPE [51] and GREEN [92] study better position embedding methods. LH-plugin [68] employs hyperbolic space to address the triangle inequality issues. BLUE [89] proposes a hierarchical architecture to model trajectories.

Trajectory Simplification. Trajectory simplification tasks can be categorized into two types, minimizing the number of remaining points given a bounding error, and minimizing the error given a fixed number of remaining points. The perpendicular Euclidean distance (PED) [57], synchronized Euclidean distance (SED) [63],

¹<https://github.com/yuanjun-liu/TrajMU>

direction-aware distance (DAD) [35], and speed-aware distance (SAD) [60] measure the point-pair-wise errors between the original and simplified trajectories. Given the number of remaining points, DPTS [53] and DOTS [4] minimize the errors. Douglas-Peucker [30] and TDMR [56] are top-down and bottom-up algorithms, respectively, to minimize the number of points with bounded error. Learning-based approaches are typically error-bounded. RLTS [75] and MARL4TS [77] utilize reinforcement learning to choose optimal points to form the simplified trajectory. S3 [22] proposes a Seq2Seq2Seq framework to encode the trajectory and then decode the simplified trajectory. RL4QDTS [76] and MLSimp [69] optimize simplification based on downstream queries.

Trajectory Map Matching. HMM [61] uses the Hidden Markov Model to map the trajectory to the most likely road route. ST-Matching [54] aims to align low sampling-rate trajectories with temporal and speed constraints. OLM [44] proposes an online algorithm for real-time map matching supported by a group of seed trajectories. SnapNet [58] is an incremental HMM algorithm for low sampling-rate cellular trajectories that filters noise points. DeepMM [23] employs an attention Seq2Seq framework to map trajectories in the latent space. LHMM [66] uses a deep-learning enhanced HMM model to map cellular trajectories using multi-relational graphs based on the co-occurrence relationship between cell towers and roads. GraphMM [49] builds a graph to mine inter-trajectory and trajectory-road correlations as well as correlations between road segments. DMM [65] developed a deep reinforcement learning-based map matching framework for cellular trajectories based on an encoder-decoder RNN network. RLOMM [9] is an online mapping algorithm based on reinforcement and contrastive learning methods alongside graph and recurrent neural networks. MMA [71] classifies trajectory points to the most likely candidate road segments with attention-enhanced point and segment embeddings. DiffMM [28] employs the encoder-diffusion architecture to match trajectories.

Trajectory Recovery. The trajectory recovery methods can be categorized into recovering in free space and recovering on the road network. CACT [33] recovers trajectories based on trajectory clustering, and SimiDTR [88] recovers trajectories based on similar trajectory searching, both focus on free space. TCVD [86] and TrajRecovery [78] are multi-modal models for recovering trajectories on the road network based on traffic camera data. TERI [12] focuses on trajectory recovery with irregular time intervals based on learnable Fourier features. RNTrajRec [13] employs multi-task learning to mine the rich spatial and temporal information of trajectories and recover trajectories on the road network. LightTR [52] recovers trajectories in free space in a federated learning scenario based on a meta-knowledge enhanced local-global training scheme. ProDiff [3] uses a diffusion model to recover trajectories in free space based on prototype guidance. TRMMA [71] employs a Seq2Seq architecture to obtain the embeddings of recovered points and then maps them to the most likely road segments.

2.2 Machine Unlearning

Given a trained model on the training set, machine unlearning [2] aims to remove the influence of the unlearning set, which is a subset of the training set. The unlearning algorithm was first introduced to solve statistical queries with an exact algorithm [2, 5], but it cannot

handle deep learning models. Retraining on the remaining set, i.e., the set of data in the training set but not in the unlearning set, is the exact algorithm for all models, but it is time-consuming. SISA [36] is an unlearning framework that partitions data and trains multiple models on each partition, enabling exact unlearning by retraining models only on specific data partitions. However, it degrades into retraining on all remaining data when the unlearning data exists in multiple partitions, and training models on small pieces of data may harm the overall performance.

The approximate unlearning algorithms are mainly assembled with basic techniques, e.g., finetuning, distillation, influence estimation, adversarial noise, and parameter perturbation. NegGrad [25] simply fine-tunes on the unlearning data by maximizing the task loss, but this can easily cause catastrophic forgetting (CF), i.e., low performance on the remaining set. BadT [17] and SCRUB [38] are teacher-student frameworks with distillation. TopK and RandomK [93] perturb parameters and then fine-tune the model on both the remaining and unlearning sets. HessianNewton [26] first computes the Hessian matrix of all parameters to estimate the influence of the unlearning set and then updates the parameters based on the Newton method, but the second-order matrix consumes significant space and is only suitable for small models. Fisher [25] estimates the influence of the unlearning set using the first-order Fisher matrix and then perturbs parameters, but simply perturbing based on the unlearning set can easily cause CF on the remaining set. SSD [24] estimates the influences of both the unlearning set and the remaining set, then adjusts the parameters directly. SFRon [32] weights parameters based on the influences of two sets and then fine-tunes the model. RandLabel [31] and NoiseLabel [85] fine-tune the model on the unlearning set with selected wrong labels. UNSIR [70] generates adversarial data for each unlearning class by maximizing the task loss, then impairs the model with adversarial data and repairs the model with a subset of the remaining set. DLFD [16] generates adversarial noise for each unlearning batch by maximizing the task loss and minimizing the optimal transport loss, then fine-tunes the model with adversarial noise on the remaining batches.

Machine unlearning literature mainly focuses on the Computer Vision field, and there are some studies investigating machine unlearning in other fields, such as graph data [11, 87], large language models [41, 83], databases [29, 37], federated learning [74, 79], and recommendation systems [18, 84].

3 Trajectory Learning Tasks

Our study aims to be general enough so that its conclusions are pertinent to different trajectory learning tasks with different models. Following, we introduce the definitions of trajectory and four trajectory learning tasks with their corresponding SOTA models.

DEFINITION 1 (TRAJECTORY). A trajectory T is defined as a time-ordered sequence of points, i.e., $T = [p_1, p_2, \dots, p_n]$, where each point $p_i = (x_i, y_i, t_i)$ records the longitude, latitude, and timestamp.

Trajectory Similarity Learning aims to represent each trajectory as a fixed-dimensional vector, such that the similarity of learned vectors represents the similarity of the trajectories. BLUE [89] is a transformer-based hierarchical encoder to represent trajectories, manipulating hierarchical trajectory patches based on the decimals of coordinates, and trained with a decoder and mean square error

(MSE) loss. TrajCL [7] optimizes a modified transformer model with contrastive learning, where the augmented trajectory is expected to have similar embedding with raw trajectory than others.

Trajectory Simplification aims to eliminate points from a trajectory T to obtain a simplified trajectory T' , such that T' has fewer points and its error is small. MLSimp [69] is a GNN-based model to simplify trajectories by evaluating the importance of points, and trained with a diffusion-based model via mutual learning. The model also utilizes the spatial and temporal range queries on the dataset to optimize the results. Since the mutual learning involves several rounds and the GNN model is retrained at each round, we treat preceding rounds as the pre-training of the diffusion model and the final round as the training of the GNN model. Moreover, the grid embeddings are pre-trained based on the original dataset D . S3 [22] is a LSTM-based Seq2Seq2Seq model, where the first sequence is the raw trajectory, the second sequence is the simplified trajectory, and the final sequence is the recovered trajectory, and the model is optimized by the reconstruction loss.

Trajectory Map Matching aims to transform a trajectory to a sequence of road edges or road nodes, where the road edges and nodes form the road network, and high matching accuracy is demanded. MMA [71] is a NN-enhanced algorithm for map matching. It matches each point with the most likely edge from the top- k candidate road network edges retrieved by the R-tree index [39], and the required sequence is obtained by linking the edges of all points sequentially. The node2vec process that yields the embeddings of road network edges is regarded as pre-training. DiffMM [28] employs a transformer to encode trajectory and a one-step diffusion model to match the point with the most likely edge from all edges retrieved by the R-tree index.

Trajectory Recovery aims to transform trajectory $T = [p_1, \dots, p_n]$ to $T' = [p'_1, \dots, p'_m]$, such that the error is minimized. Generally, T is sparser than T' , i.e., $n < m$, or T has missing parts, i.e., $\exists i, j \in [2, n], p_{i+1}.t - p_i.t \gg p_{j+1}.t - p_j.t$. TRMMA [71] is a NN-enhanced algorithm for trajectory recovery based on map matching. It computes the optimal road network edge of each recovered point p' among map-matched road network edges of T , then recovers the positions. The training of MMA on the original dataset to obtain the embeddings of road segments and map-matching results is regarded as pre-training. ProDiff [3] first classifies trajectory to a prototype, then reconstructs trajectory with a diffusion model guided by the prototype and endpoints solely.

4 Machine Unlearning Methods

Given a learning algorithm $\mathcal{A}$, the original model θ_0 is trained on the original training set D , i.e., $\theta_0 \leftarrow \mathcal{A}(D)$. A well-known learning algorithm is minimizing the task-relevant loss $\mathcal{L}$ by gradient descent, i.e., $\theta_0 \leftarrow \theta_* - \frac{\alpha}{|D|} \sum_{x \in D} \nabla \mathcal{L}(x)$, where α is the learning rate, and θ_* represents the randomly initialized parameters.

DEFINITION 2 (MACHINE UNLEARNING). *Given the unlearning set $D_u \subset D$ and the original model parameters θ_0 , machine unlearning aims to design an unlearning algorithm $\mathcal{U}$ such that the unlearned parameters θ_u are similar to the retrained parameters θ_r , where $\theta_u \leftarrow \mathcal{U}(D_u | \theta_0)$, $\theta_r \leftarrow \mathcal{A}(D_r)$, and $D_r = D \setminus D_u$ is the remaining set.*

Previous studies [24, 38] also describe unlearning the unlearning set D_u as forgetting the forgetting set. Similarly, the remaining set is also referred to as the retain set.

Below, we introduce ten unlearning methods that are universal to different learning tasks. Each of these offers a different procedure based on the remaining set D_r and/or the unlearning set D_u , and is mainly assembled with basic techniques, e.g., finetuning, distillation, influence estimation, and parameter perturbation.

Retrain, the ‘oracle’ solution, retrains the model on the remaining set, i.e., $\theta_r \leftarrow \mathcal{A}(D_r)$. However, this is time-consuming, whereas unlearning algorithms are expected to be more efficient.

FineTune simply fine-tunes the model on the remaining set D_r for a small number of epochs, steering the model towards forgetting the unlearning set D_u , i.e., $\theta_u \leftarrow \theta_0 - \frac{\alpha}{|D_r|} \sum_{x \in D_r} \nabla \mathcal{L}(x)$.

NegGrad [25] fine-tunes the model on the remaining set D_r with a negated gradient, attempting to unlearn D_u by maximizing the loss via gradient ascent on that data and ‘undoing’ the gradient descent process that the network had previously undergone, i.e., $\theta_u \leftarrow \theta_0 + \frac{\alpha}{|D_r|} \sum_{x \in D_r} \nabla \mathcal{L}(x)$.

BadT [17] employs the original model as a good teacher and a randomly initialized model as a bad teacher to train a new student model on the remaining set D_r . This is done by minimizing the task loss and the distillation loss with the good teacher while maximizing the distillation loss with the bad teacher, i.e., $\theta_u \leftarrow \theta_* - \frac{\alpha}{|D_r|} \sum_{x \in D_r} \nabla (\mathcal{L}(x) + \gamma \mathcal{L}_d(x | \theta_0)) + \frac{\alpha \gamma}{|D_r|} \sum_{x \in D_r} \nabla \mathcal{L}_d(x | \theta_*)$, where $\mathcal{L}_d$ is the distillation loss and γ is its weight.

SCRUB [38] employs the original model as the teacher model and fine-tunes the model by maximizing the task loss and the distillation loss on the unlearning set D_u , and minimizing the two losses on the remaining set D_r , iteratively, i.e., $\theta_u \leftarrow \theta_0 + \frac{\alpha}{|D_u|} \sum_{x \in D_u} \nabla (\mathcal{L}(x) + \gamma \mathcal{L}_d(x | \theta_0)) - \frac{\alpha}{|D_r|} \sum_{x \in D_r} \nabla (\mathcal{L}(x) + \gamma \mathcal{L}_d(x | \theta_0))$.

For the BadT and SCRUB methods, the distillation loss is set as the KL loss for the MMA and TRMMA models, since their loss is classification loss, and set as the MSE loss for others.

GDRGMA [45] observes the conflict between the gradients of the remaining data and unlearning data, which hinders the unlearning effect. It dynamically modifies the directions and magnitudes of gradients for remaining and unlearning batches, such that the cosine value of two gradients is not less than zero, then updates the model with a gradient composed of $(1 - \lambda) \times$ modified gradient of the remaining batch $+ \lambda \times$ modified gradient of the unlearning batch, where λ is a self-tuning parameter.

TopK and RandomK [93] perturb a fraction of model parameters, where RandomK picks randomly, while TopK selects most sensitive parameters. Then, they finetune the model by minimizing the task loss on the remaining batches and the JS loss on the unlearning batches with the original model.

The sensitivity of parameters is measured with the Fisher information. Given a dataset D and the task loss function $\mathcal{L}$, the Fisher information value of each parameter $\theta^{(i)} \in \theta$ is computed as

$$F_i(D) = \frac{1}{|D|} \sum_{x \in D} \left(\frac{\partial}{\partial \theta^{(i)}} \mathcal{L}(x | \theta) \right)^2$$

SFRon [32] first computes parameter weights by measuring the Fisher information on the remaining set D_r and unlearning set D_u , and tunes the parameters θ' having more Fisher information on the

unlearning set D_u , i.e., $\theta' = \{\theta^{(i)} | F_i(D_u) > F_i(D_r), \theta^{(i)} \in \theta\}$. Then, it iteratively performs gradient ascent adjusted by weights on the unlearning batch and gradient descent on the remaining batch.

SSD [24] adjusts the parameters directly by suppressing the parameters that have higher Fisher information on the unlearning set D_u , i.e., $\theta^{(i)} = \min(\lambda F_i(D)/F_i(D_u), 1)\theta^{(i)}$ if $F_i(D_u) > \alpha F_i(D)$ else $\theta^{(i)}$, where λ and α are hyper-parameters.

Besides the above methods, there is a wide range of unlearning methods designed for specific tasks or requiring particular conditions, that *can hardly be applied to trajectory data directly, and how to migrate them is an open problem*. UNSIR [70], RandLabel [31], and NoiseLabel [85] are suitable for multi-class classification, but the BLUE and MLSimp models do not use classification loss, and the MMA and TRMMA models depend on binary classification, which degrades the core adversarial step to simply maximizing the task loss. DLF [16] learns adversarial noise, but since trajectory shapes are highly variable, the learned adversarial noise on specific trajectories can hardly be applied to others. Moreover, the trajectory models have complex and non-differentiable processes. Specifically, the BLUE model builds hierarchical patches for trajectories, the MLSimp model transforms trajectories into sequences of grids, and the TRMMA model matches trajectories to sequences of road segments. These operations block the gradients to adversarial noise.

5 Measuring the Impact of Unlearning

As mentioned earlier, evaluating unlearning is an open problem in ML literature. Besides running time, two aspects are mainly measured. The Membership Inference Attack (MIA) [10, 67] evaluates the privacy status of the model, and task metrics measure the performance of the model on downstream tasks.

MIAs. The core idea behind MIA is to train a binary classifier to distinguish between members, i.e., the remaining set D_r , and non-members, i.e., the unseen validation set D_v . Then, the classifier is employed to infer the membership of the unlearning set D_u . The MIA value of a trajectory T is 1 if T is classified as a member else 0. The MIA score is computed as the averaged classification result on the unlearning set D_u , i.e., $\frac{1}{|D_u|} \sum_{T_i \in D_u} MIA(\theta, T_i)$, where $MIA(\theta, T_i)$ is the classification result of trajectory T_i .

We focus on the common black-box attack setting where the attacker observes only the outputs of the model. Thus, we adopt two attacks used in unlearning literature and apply them to trajectory models, the confidence-based attack [24], where the MIA classifier is trained on the output vectors, and the sample-based attack [72], where the MIA classifier is trained on the output trajectories. For the trajectory similarity and map-matching tasks that yield the representations and the probabilities, we use the confidence-based attack. Specifically, for the map-matching task, we take the top- k probabilities of each point and treat each point as an individual sample, where k is set as 10 following [71]. For the trajectory simplification and recovery tasks that output the sampled and recovered trajectories, we use the sample-based attack.

Task Metrics. It is worth noting that trajectory learning models are designed with robustness. For the trajectory similarity task, models can infer on unseen datasets directly with a little performance drops [51, 89]. For the trajectory simplification task, models adjust results based on queries on the dataset [69, 76], utilizing

information from unseen data. For map-matching and trajectory recovery tasks, models consider nearby candidates [9, 13, 71], ensuring that matching results are not drastically incorrect. Thus, we evaluate downstream tasks on not only unlearning set D_u and remaining set D_r , but also an unseen validation set D_v .

For the similarity task, the ranking and k -nn tasks quantify the quality of similarity. Given the query set D_q and database set D_d , the smaller the rank of query trajectory $T_i \in D_q$ in $D_d \cup D'_q$, the better the learned similarity, where D'_q is the noised query set, and the noised trajectory T'_i is regarded as the most similar trajectory of query trajectory T_i . Let s_{ij} be the similarity of trajectory $T_i \in D_q$ and $T_j \in D_d$ and $s_{i'j'}$ be the similarity of trajectory T_i and its most similar trajectory $T'_{i'}$, γ_i be the rank of $s_{i'j'}$ in $(s_{i1}, s_{i2}, \dots, s_{i|D_d|})$. The mean rank (MR) is defined as $\frac{1}{|D_q|} \sum_{i=1}^{|D_q|} \gamma_i$. Given the query set D_q , noised query set D'_q , and database set D_d , the higher hit ratio of T'_i in $knn(T_i, D_d)$ for a query trajectory $T_i \in D_q$, the better the learned similarity. The hit ratio at the top- k results, i.e., $HR@k$, is computed as $\frac{1}{|D_q|} \sum_{i=1}^{|D_q|} \mathbb{1}T'_i \in knn(T_i, D_d)$, where $\mathbb{1}T'_i \in knn(T_i, D_d)$ is 1 if the most similar trajectory T'_i of T_i is retrieved, else 0. $MR \geq 1$ and $HR@k \in [0, 1]$, and lower MR and higher HR results mean better performance. The query set is set as the unlearning set D_u , the remaining set D_r , and the validation set D_v , respectively.

For the simplification task, the SED evaluates simplification error, and the F1 measures the performance on range query task. Given trajectory $T = [\dots, p_i, \dots]$ and its simplified trajectory $T' = [\dots, p'_j, \dots]$, SED measures the maximal distance of each point $p_i \in T$ from its synchronized point q_i having the same time and located on T' , i.e., $\max_{p_i \in T} \sqrt{(p_i.x - q_i.x)^2 + (p_i.y - q_i.y)^2}$, where $q_i.x = p'_j.x + \delta(p'_{j+1}.x - p'_j.x)$, $q_i.y = p'_j.y + \delta(p'_{j+1}.y - p'_j.y)$, $\delta = \frac{p_i.t - p'_j.t}{p'_{j+1}.t - p'_j.t}$, and $p'_j.t \leq p_i.t \leq p'_{j+1}.t$. $SED \geq 0$ and lower SED is better. While SED focus on local distance of points and simplified segments, ITS [50] measures the distance of whole Trajectory efficiently. The two trajectories are first converted into interpolated trajectories $\hat{T} = [\dots, \hat{p}_i, \dots, \hat{p}_h]$ and $\hat{T}' = [\dots, \hat{q}_i, \dots, \hat{q}_h]$, where h points distribute evenly on the arc, and each point pair $\hat{p}_i$ and $\hat{q}_i$ have the same arc length ratio on interpolated trajectories. Then, the ITS is calculated as $\sqrt{\frac{1}{h} \sum_{i=1}^h \|\hat{p}_i - \hat{q}_i\|^2}$, $ITS \geq 0$ and lower ITS is better. Moreover, for query-driven simplification, the F1 value evaluates the performances on downstream tasks.

For the map matching task, the Acc and F1 metrics are employed. Given a trajectory T , let S be the matched road segments of the ground truth and S' be the matched road segments of the model, the F1 score of the map matching task is computed as $F1(S, S')$. And the Acc counts the number of identical segments, i.e., $Acc = (\sum_{1 \leq i \leq |S|} \mathbb{1}S_i = S'_i) / |S|$, where $\mathbb{1}S_i = S'_i$ is 1 if the i -th matched segments are the same. $Acc \in [0, 1]$ and higher Acc is better.

For the recovery task, the MAE measures recovery error. Given the recovered trajectory $T' = [\dots, p'_j, \dots]$ and its ground truth $T = [\dots, p_i, \dots]$, MAE is computed as the average distance of all point pairs, i.e., $\frac{1}{|T|} \sum_{p_i \in T} \sqrt{(p_i.x - p'_j.x)^2 + (p_i.y - p'_j.y)^2}$. MAE ≥ 0 and lower MAE is better. Moreover, the TRMMA model matches the to-be-recovered point to the candidate road segments, and ProDiff regards a recovered point as correct if its distance to real point is less than a threshold, so the Acc evaluates the accuracy.

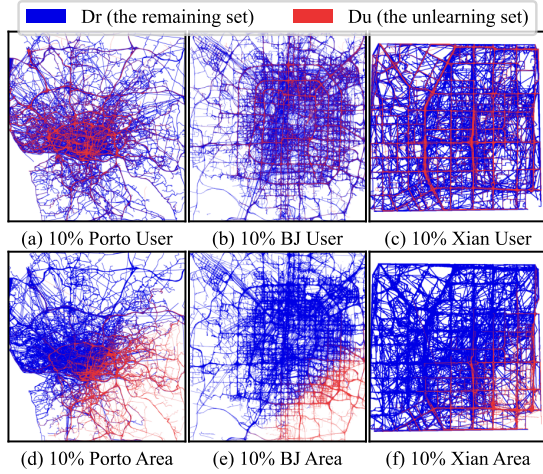


Figure 1: Dataset Visualization of the unlearning set D_u and the remaining set D_r when unlearning 5% data. (a) the User scenario of the Porto dataset, (d) the Area scenario of the Porto dataset, (b) the User scenario of the BJ dataset, (e) the Area scenario of the BJ dataset, (c) the User scenario of the Xian dataset, (f) the Area scenario of the Xian dataset.

6 Experimental Evaluation

We empirically evaluate and analyze the aforementioned ten unlearning algorithms on three real-world datasets, in the context of several trajectory tasks and evaluation metrics.

6.1 Experimental Setup

Dataset. We experiment on three real-world trajectory datasets. Porto [59] contains over one million trajectories collected in Porto, Portugal, from 1 July 2013 to 30 June 2014 by 442 drivers, BJ [64] contains over one million trajectories collected in Beijing, China, from 2 May 2009 to 25 May 2009 by 15,920 users, and Xian [19] contains over three million trajectories collected in Xian, China, from 1 Oct. 2016 to 31 Oct. 2016 by 535,828 users. All datasets can be downloaded from our code repository. We randomly select 100,000 trajectories as the training set to train the original model and two sets for testing and validating during training, each of size 20,000. Then, the two sets are combined as the unseen validation set D_o to execute the MIA, and the training set is split into two distinct sets, i.e., the remaining set D_r and the unlearning set D_u . Specifically, for MLSimp, the size of the training set is set to 1,000 following [69], and the sizes of other sets are reduced proportionately.

Unlearning Scenarios. We consider two unlearning scenarios: Unlearn User, where a group of users requests to unlearn all their trajectories, and Unlearn Area, where a group of trajectories closest to a certain area are considered outdated and unlearned. Each scenario contains three unlearning ratios: 10% 20% and 30%, and we report the averaged results on three ratios.

Given the unlearning ratio r_u , the training set D , and the target area Φ , the Area scenario unlearns trajectories D_u that are closest to the area, i.e., $dis(T_i, \Phi) \geq dis(T_j, \Phi)$, $T_i \in D_r$, $T_j \in D_u$, $|D_u| = |D|r_u$, $D_r = D \setminus D_u$, $0 \leq r_u \leq 1$. This scenario simulates a situation where a certain area changes its function or the trajectory patterns

passing through it change, so the learned model is expected to unlearn the outdated information. $dis(T, \Phi)$ computes the distance of trajectory T to the area Φ , and we take the averaged distance of all points to the area center. Given the unlearning ratio r_u , the training set D and the user set Ψ with their trajectories $\mathcal{Y}_i$, i.e., $\mathcal{Y}_i \subset D$, $i \in \Psi$, the User scenario unlearns trajectories D_u belonging to Ψ_u , i.e., $D_u = \bigcup_{i \in \Psi_u} \mathcal{Y}_i$, $|\Psi_u| = |\Psi|r_u$, $D_r = D \setminus D_u$, $0 \leq r_u \leq 1$. This scenario models privacy-driven deletion requests, where users exercise their right to remove their data. Specifically, a fraction r_u of users request the deletion of all their trajectories.

We visualize the remaining set D_r and the unlearning set D_u in Figure 1. It is obvious that the data distributions of the two unlearning scenarios, i.e., the User and Area, are distinct. The User scenario is more random than the Area unlearning, as users are dispersed throughout the city. In contrast, the D_u of the Area scenario is more centralized, and we chose areas at the edge.

Compared Methods. We compare the aforementioned ten unlearning methods based on two groups of models, the default group contains BLUE, MLSimp, MMA, and TRMMA models, and the second group includes TrajCL, S3, DiffMM, ProDiff models, for trajectory similarity computation, simplification, map matching, and recovery tasks, respectively. For the unlearning methods, the parameters are set as their default, and the epochs are set as the one where the unlearning metric results do not change anymore. For the BadT and SCRUB methods, the distillation loss is set as the KL loss for the MMA and TRMMA models, since their loss is classification loss, and set as the MSE loss for others. For the TopK and RandomK algorithms, the JS loss here is replaced with the MSE loss for BLUE and MLSimp models since they do not rely on classification loss. Following [93], the weight of the distilling loss is set to 0.1. For the trajectory learning models, they are trained with default configurations, including training epochs, learning rate, batch size, etc. The trajectory length of simplification task is set to 500 for Porto, 110 for BJ, and 400 to Xian to ensure adequate trajectory numbers, and the simplification rate is set to 1% for Porto, 5% for BJ, and 1% for Xian to ensure the simplified trajectories are valid.

Evaluations. The running time evaluates the efficiency of unlearning methods. For MIA, the similarity and map matching tasks use the confidence-based MIA, while the simplification and recovery tasks utilize the sample-based method. For task metrics, we employ two groups of metrics. The default metrics are MR for similarity, SED for simplification, F1 for map matching, and MAE for recovery. Another group of metrics are HR@10 for similarity, F1 for simplification, Acc for map matching, and Acc for recovery.

We treat the results of Retrain as the golden standard for task metrics and MIAs, and shorter running time is preferred. Since the metrics have different ranges and preference, e.g., $MR \geq 0$ and lower MR is better, and $F1 \in [0, 1]$ and higher F1 is better, we employ the SimScore to analyze the similarity of between the results of Retrain $x_{gt} \geq 0$ and the results of another unlearning method $x_u \geq 0$. The SimScore of two results x_u, x_{gt} is calculated as

$$\text{SimScore}(x_u, x_{gt}) = \begin{cases} 1 - |x_u - x_{gt}|, & \text{metric range} \subset [0, 1] \\ \frac{\min(x_u, x_{gt})}{\max(x_u, x_{gt})}, & \text{otherwise.} \end{cases}$$

SimScore $\in [0, 1]$, and higher score indicates that the unlearning method is better aligned with the Retrain.

Table 1: Summarized Task Results on Model Group 1

Method	Metric Group 1															Metric Group 2		
	Porto					BJ					Xian					Porto	BJ	Xian
	Sim.	Simp.	Map.	Rec.	Rk.	Sim.	Simp.	Map.	Rec.	Rk.	Sim.	Simp.	Map.	Rec.	Rk.	Rk.	Rk.	Rk.
Retrain	1.875	0.038	0.870	100.2	-	8.027	0.013	0.827	254.9	-	4.206	0.019	0.956	78.1	-	-	-	-
FineTune	1.633	0.039	0.886	115.2	4	36.074	0.013	0.834	254.7	4	2.447	0.019	0.968	107.3	4	4	5	4
NegGrad	5791.420	0.039	0.110	402.4	9	5087.995	0.013	0.504	713.6	9	10450.453	0.019	0.810	367.7	9	9	9	9
BadT	4.735	0.039	<u>0.887</u>	115.5	5	40.597	0.013	0.836	256.2	5	6.432	0.019	0.967	110.2	3	5	4	3
SCRUB	2.867	0.038	0.685	391.8	7	74.563	0.013	0.631	659.6	8	<u>5.991</u>	0.019	0.850	336.8	6	7	7	6
GDRGMA	1.798	0.038	0.907	99.0	1	<u>20.227</u>	0.013	<u>0.830</u>	<u>255.6</u>	1	2.849	0.019	0.971	76.9	1	1	2	1
TopK	4566.998	<u>0.038</u>	0.887	111.7	6	4086.008	<u>0.013</u>	0.832	264.7	6	4139.831	0.019	<u>0.967</u>	111.6	7	8	8	8
RandomK	2.031	0.039	0.893	104.8	3	20.909	0.013	0.833	259.5	3	3.128	0.019	0.970	95.7	<u>2</u>	3	3	<u>2</u>
SFRon	70.556	0.039	0.759	306.5	8	22.333	0.013	0.661	609.8	7	41.235	<u>0.019</u>	0.865	292.6	8	6	6	7
SSD	1.820	0.039	0.902	<u>103.4</u>	<u>2</u>	20.125	0.013	0.829	258.7	<u>2</u>	367.371	0.019	0.967	<u>82.2</u>	5	<u>2</u>	1	5

Table 2: Summarized Task Results on Model Group 2

Method	Metric Group 1															Metric Group 2		
	Porto					BJ					Xian					Porto	BJ	Xian
	Sim.	Simp.	Map.	Rec.	Rk.	Sim.	Simp.	Map.	Rec.	Rk.	Sim.	Simp.	Map.	Rec.	Rk.	Rk.	Rk.	Rk.
Retrain	4.990	0.035	0.868	920.0	-	26.657	0.014	0.777	1048.5	-	114.869	0.019	0.949	376.7	-	-	-	-
FineTune	<u>3.901</u>	0.034	0.877	<u>824.8</u>	1	15.508	0.013	0.819	996.5	1	24.323	0.019	0.960	<u>244.3</u>	3	1	1	<u>2</u>
NegGrad	32636.145	0.045	0.258	11597.5	9	42649.916	0.018	0.194	16713.4	9	34994.464	0.024	0.335	5684.8	9	9	9	9
BadT	3.933	<u>0.034</u>	<u>0.875</u>	728.9	<u>2</u>	<u>14.743</u>	0.013	0.821	903.3	<u>2</u>	32.863	0.018	0.960	217.4	4	3	3	3
SCRUB	43652.734	0.042	0.247	726.5	8	41946.240	0.019	0.236	911.6	7	32779.200	0.025	0.303	216.1	8	8	8	8
GDRGMA	3.280	0.033	0.882	727.1	3	13.750	0.013	<u>0.808</u>	911.3	3	15.203	0.018	0.963	217.8	5	4	<u>2</u>	5
TopK	127.123	0.035	0.874	12237.8	6	2247.377	0.013	0.599	18919.5	8	215.728	<u>0.019</u>	<u>0.956</u>	5842.6	6	6	7	7
RandomK	20.986	0.034	0.882	2672.0	5	138.064	<u>0.013</u>	0.807	2478.1	5	<u>104.039</u>	0.018	0.963	1215.6	<u>2</u>	5	5	4
SFRon	747.069	0.042	0.311	727.5	7	1246.774	0.017	0.235	<u>912.1</u>	6	552.345	0.023	0.354	217.0	7	7	6	6
SSD	22.326	0.033	0.879	861.5	4	110.163	0.013	0.697	1324.4	4	108.766	0.018	0.954	377.7	1	<u>2</u>	4	1

Table 3: Summarized MIA Results among Scenarios

Method	Model Group 1										Model Group 2									
	User					Area					User					Area				
	Sim.	Simp.	Map.	Rec.	Rk.	Sim.	Simp.	Map.	Rec.	Rk.	Sim.	Simp.	Map.	Rec.	Rk.	Sim.	Simp.	Map.	Rec.	Rk.
Retrain	0.502	0.659	0.499	0.492	-	0.102	0.409	0.499	0.091	-	0.499	0.653	0.501	0.641	-	0.056	0.424	0.487	0.687	-
FineTune	<u>0.500</u>	<u>0.651</u>	0.500	0.507	4	0.126	0.390	0.502	<u>0.088</u>	1	0.524	0.603	0.501	0.659	7	0.070	0.484	0.499	<u>0.677</u>	<u>2</u>
NegGrad	0.498	0.672	0.501	0.489	<u>2</u>	0.497	0.431	0.500	0.127	9	0.501	0.579	0.499	0.474	9	0.490	0.437	0.498	0.574	9
BadT	0.503	0.660	<u>0.499</u>	0.511	1	0.104	0.487	0.499	0.088	3	0.520	0.693	0.499	0.672	8	0.061	0.492	0.498	0.681	1
SCRUB	0.499	0.742	0.500	0.516	9	<u>0.093</u>	0.510	0.477	0.099	6	0.501	0.632	0.500	<u>0.637</u>	<u>2</u>	0.501	<u>0.470</u>	0.500	0.666	8
GDRGMA	0.500	0.716	0.502	<u>0.496</u>	8	0.092	0.483	0.500	0.101	5	0.520	0.629	<u>0.501</u>	0.685	6	0.069	0.482	0.500	0.698	3
TopK	0.499	0.635	0.501	0.498	6	0.376	0.393	0.499	0.087	8	0.496	<u>0.674</u>	0.500	0.666	4	0.106	0.483	0.502	0.591	7
RandomK	0.500	0.709	0.499	0.501	7	0.084	0.456	<u>0.499</u>	0.084	<u>2</u>	0.505	0.608	0.500	0.651	5	0.046	0.506	0.501	0.666	5
SFRon	0.499	0.668	0.501	0.501	3	0.077	0.487	0.430	0.118	7	<u>0.501</u>	0.689	0.500	0.647	3	0.067	0.489	<u>0.498</u>	0.623	6
SSD	0.496	0.646	0.501	0.487	5	0.133	0.451	0.493	0.098	4	0.516	0.654	0.502	0.639	1	<u>0.049</u>	0.488	0.494	0.663	4

6.2 Experimental Results

Limited by space, we show the summarized result below.

Summary Results on Four Tasks. Table 1 and Table 2 show the results on two model groups, and each model group use two metric groups, where the default metric group evaluates the MR, SED, F1, and MAE values, and the second metric group computes the HR@10, ITS, Acc, and Acc values, for similarity computation, simplification, map matching, and recovery tasks, respectively. Each unlearning algorithm has summarized ranks on difference datasets, models, and metrics, calculated by average SimScore on four tasks.

On the whole, unlearning algorithms have almost the same ranks among different metrics on the same model, and most algorithms have similar ranks among different models. FineTune,

BadT, GDRGMA, RandomK, and SSD perform well on all models and metrics, and NegGrad, SCRUB, TopK, and SFRon underperform.

Analysis on MIA. Table 3 presents the MIA results on four tasks among two unlearning scenarios, i.e., User and Area, and two model groups, and ranks of the SimScore. As shown in Figure 1, the unlearning set D_u of Area is more centralized and distinct from the remaining set D_r , so the MIA values of Area are much lower. Conversely, the distributions of the unlearning and remaining sets of User are similar, the remaining set contains the information of the unlearning set, so the MIA values of User are higher.

FineTune, BadT, and SSD perform well on three settings cross different models and scenarios, GDRGMA and RandomK fit Area, SFRon suits User, and others are unstable on various settings.

Table 4: Summarized Results among Unlearning Batches

Method	Unlearn in One Batch										Unlearn in Two Batch									
	Task					MIA					Task					MIA				
	Sim.	Simp.	Map.	Rec.	Rk.	Sim.	Simp.	Map.	Rec.	Rk.	Sim.	Simp.	Map.	Rec.	Rk.	Sim.	Simp.	Map.	Rec.	Rk.
Retrain	4.703	0.023	0.885	144.4	-	0.302	0.534	0.499	0.292	-	4.703	0.023	0.885	144.4	-	0.302	0.534	0.499	0.292	-
FineTune	13.385	<u>0.023</u>	<u>0.896</u>	159.1	3	0.313	0.521	0.501	0.297	<u>2</u>	23.709	0.023	0.898	163.0	5	0.301	0.576	0.501	0.292	3
NegGrad	7109.956	0.023	0.475	494.6	9	0.498	0.552	0.501	0.308	9	7399.928	<u>0.023</u>	0.515	508.2	9	0.500	0.577	0.500	0.335	9
BadT	17.255	0.023	0.897	160.6	4	0.304	0.573	<u>0.499</u>	0.299	3	11.064	0.023	0.899	160.8	3	<u>0.300</u>	0.594	0.498	0.298	4
SCRUB	27.807	0.023	0.722	462.7	8	<u>0.296</u>	0.626	0.488	0.308	7	10.222	0.023	0.829	405.9	7	0.295	0.580	0.476	0.303	6
GDRGMA	8.291	0.023	0.903	143.8	1	0.296	0.600	0.501	0.298	5	8.291	0.023	0.903	143.8	1	0.289	0.567	0.499	0.320	5
TopK	4264.279	0.023	0.895	162.7	6	0.437	0.514	0.500	<u>0.292</u>	8	5560.713	0.023	<u>0.891</u>	173.5	6	0.465	0.530	0.501	<u>0.290</u>	8
RandomK	<u>8.689</u>	0.023	0.899	153.3	2	0.292	0.582	0.499	0.292	4	<u>9.221</u>	0.023	0.898	<u>153.4</u>	<u>2</u>	0.305	<u>0.548</u>	<u>0.498</u>	<u>0.298</u>	1
SFRon	44.708	0.023	0.762	403.0	7	0.288	0.578	0.465	0.309	6	18.062	0.023	0.721	424.9	8	0.288	0.565	0.472	0.319	7
SSD	129.772	0.023	0.899	<u>148.1</u>	5	0.314	<u>0.549</u>	0.497	0.293	1	9.883	0.023	0.884	220.4	4	0.305	0.569	0.498	0.295	<u>2</u>

Table 5: Correlations of Road Network Topological Features and Unlearn Results on BJ Dataset

Method	Task										MIA									
	Node Coverage				Node Density				Avg. Node Degree				Node Coverage				Node Density			
	Sim.	Simp.	Map.	Rec.	Sim.	Simp.	Map.	Rec.	Sim.	Simp.	Map.	Rec.	Sim.	Simp.	Map.	Rec.	Sim.	Simp.	Map.	Rec.
FineTune	0.53	-0.04	0.85	0.68	0.80	-0.60	0.54	0.39	0.53	-0.04	0.85	0.68	0.80	-0.60	0.54	0.39	0.58	-0.05	0.57	0.35
NegGrad	0.52	0.09	0.36	0.43	0.86	0.25	-0.25	0.83	0.52	0.09	0.36	0.43	0.86	0.25	-0.25	0.83	0.57	0.08	0.05	0.19
BadT	0.50	-0.65	0.59	0.41	0.62	0.37	0.16	-0.35	0.50	-0.65	0.59	0.41	0.62	0.37	0.16	-0.35	0.51	-0.65	0.26	0.09
SCRUB	0.50	-0.21	-0.43	-0.37	0.90	0.15	0.74	0.72	0.50	-0.21	-0.43	-0.37	0.90	0.15	0.74	0.72	0.50	-0.13	-0.42	-0.67
GDRGMA	0.49	-0.16	0.81	0.57	0.88	0.00	0.42	-0.62	0.49	-0.16	0.81	0.57	0.88	0.00	0.42	-0.62	0.54	-0.01	0.52	0.22
TopK	-0.22	-0.05	0.83	-0.30	0.97	0.01	0.29	0.57	-0.22	-0.05	0.83	-0.30	0.97	0.01	0.29	0.57	0.08	-0.08	0.55	0.04
RandomK	0.49	-0.02	0.87	0.03	0.78	0.51	0.36	0.61	0.49	-0.02	0.87	0.03	0.78	0.51	0.36	0.61	0.52	0.03	0.61	-0.09
SFRon	0.60	-0.08	-0.24	0.72	0.91	0.18	0.37	0.93	0.60	-0.08	-0.24	0.72	0.91	0.18	0.37	0.93	0.62	0.12	0.11	0.58
SSD	0.50	-0.45	0.63	0.67	0.88	0.04	0.72	-0.44	0.50	-0.45	0.63	0.67	0.88	0.04	0.72	-0.44	0.54	-0.30	0.30	0.41

Table 6: Summarized Speedup Results

Method	Sim.	Simp.	Map.	Rec.	Rank
FineTune	11	114	9	15	5
NegGrad	25	288	23	37	3
BadT	2	28	3	2	9
SCRUB	3	28	3	2	8
GDRGMA	<u>29</u>	460	38	53	1
TopK	9	98	7	4	6
RandomK	23	248	16	5	4
SFRon	7	77	6	10	7
SSD	30	<u>322</u>	<u>27</u>	<u>44</u>	<u>2</u>

Analysis on Unlearn Batches. While previous analyses unlearn the whole unlearning set D_u in one batch, Table 4 investigates the sequential unlearning setting. We divide the unlearning set into two equal and non-overlapping batches, apply unlearning sequentially twice on default models, and evaluate by default metrics.

Method rankings remain largely consistent with the single batch results, indicating that the evaluated methods do not suffer severe cumulative drift over two batches.

Case Study on Topological Features of Road Network. Table 5 demonstrates a quantitative analysis of how specific topological features affect the stability of different unlearning algorithms. We compute the node coverage, node density, and averaged node degree of the unlearning set D_u , i.e., $\tau_u^c, \tau_u^p, \tau_u^d$, and of the remaining set D_r , i.e., $\tau_r^c, \tau_r^p, \tau_r^d$. For each feature, each task, and each unlearning algorithm, we evaluate the pearson correlation between $|\tau_u - \tau_r|$ and $1 - \text{SimScore}(x_u, x_{gt})$, among all unlearning scenarios and unlearning ratios on BJ dataset. The value $PC \in [-1, 1]$, and $PC < 0$ and $PC > 0$ indicate negative and positive linear relations, respectively.

Results from the similarity, map matching, and recovery tasks reveal a strong correlation between unlearning performance and road network topological features. Specifically, as the topological gap $|\tau_u - \tau_r|$ between the unlearning set and the remaining set increases, the SimScore achieved by the unlearning algorithm decreases, indicating a greater deviation from the Retrain.

Analysis on Efficiency. Unlearning algorithms are motivated by the desire to avoid the high costs of retraining. We thus evaluate the efficiency of the approximate unlearning algorithms studied. We record the speed-up of each algorithm over Retrain for our four trajectory learning tasks and their corresponding models, and report the averaged speed-up results over unlearning scenarios, datasets, and unlearning ratios. Table 6 shows the results of unlearning efficiency. GDRGMA has the highest speedup, which tunes on a small fraction of data and runs a few epochs. SSD requires no tuning and is more efficient than others. NegGrad tunes on the unlearning set merely and is faster than following methods that tune on the remaining set. Compared to RandomK, TopK needs more time to find sensitive parameters. The BadT algorithm is the most time-consuming unlearning algorithm, since it contains three models, i.e., the good teacher model, the bad teacher model, and the student model, and finetunes on the full remaining set.

7 Learned Lessons

While most questions have been answered in specific sections, several questions need to be reinforced.

Data Distributions and Their Effects. We focus on two scenarios, the User scenario, which unlearns all trajectories belonging to a group of users, and the Area scenario, which unlearns a group of trajectories near a specific area. Previous analyses have shown

that these two scenarios have different data distributions and produce disparate results. *For the User scenario, the distributions of the remaining set D_r and the unlearning set D_u are similar*, as users live and act all over the city, leading to spatial distributions similar to selecting and unlearning trajectories randomly. Consequently, the remaining set contains the information about the unlearning set, and the MIA results are generally around 0.5. In contrast, *the Area scenario, where trajectories are separated according to spatial location, has more distinguishable distributions between the remaining and unlearning sets*. The MIA results saw larger gaps. What’s more, *trajectory data share the same space*. Generally, the longitude and latitude ranges and the road network of the unlearning set D_u and remaining set D_r intersect. It is possible that a point p exists in both datasets, i.e., $p \in T_u, p \in T_r, T_u \in D_u, T_r \in D_r$. Thus, even if the model has not seen a trajectory T_u , the spatio-temporal information of the trajectory may be contained in D_r and learned by the model.

Therefore, *trajectory data prefer gentler unlearning algorithms to align similar distributions*. NegGrad and SFRon algorithms employ gradient ascent on the unlearning set, and TopK modifies most sensitive parameters based on the unlearning set, these operations damage parameters dramatically, so they generally underperform in Tables 1 to 3. RandomK modifies less sensitive parameters and is better than TopK, GDRGMA reduces the gradients conflict, and FineTune simply tunes on the remaining set, they are less aggressive and obtained moderate task ranks.

Moreover, *the road networks also bring shifts in data distribution*. Figure 1 shows that the road networks of BJ and Xian are more regular than that of Porto, so their simplification errors are lower. Also, the road networks of Porto and Xian are sparser, so their map matching accuracies are higher, and the recovery errors are lower.

Characteristics of Different Learning Tasks. Distinct characteristics complicate unlearning and evaluation. For the trajectory similarity task, models are designed with robustness [51, 89], they can provide high-quality results on unseen datasets directly, and naturally on the unlearning set. For the trajectory simplification task, the MLSimp can employ queries to adjust simplification results, and the pre-trained mutual learning model and grid embeddings contain the information about the unlearning set.

The trajectory map matching and trajectory recovery tasks rely on the road network and match points to candidate segments. First, the embeddings of the segments of the road network are pre-trained and fixed, and matching is enhanced by the R-tree index to improve the probability of choosing the truth, so models are likely to maintain performance since the road network remains the same. Second, as mentioned in Table 5, trajectory data collected on the road network depend on topological structure. Third, for shortest-path based recovery model TRMMA, if one matched road segment changes, the shortest paths with its adjacent segments change, then the recovered trajectory changes intensely. Thus, recovered trajectories may change significantly if even one segment is mismatched.

8 Open Problems

There are several unsolved problems and research directions for unlearning on trajectory data.

How to migrate other unlearning algorithms? As mentioned before, there is a group of unlearning algorithms that cannot be

applied to trajectory models directly. Adversarial-based algorithms [16, 70] train noises or samples to antagonize the unlearning set D_u , then train the model with the noises or samples to stimulate unlearning D_u . However, on the one hand, trained noise with a fixed shape is not practical for trajectories with various shapes. On the other hand, trajectory models process trajectories complexly. Specifically, BLUE builds hierarchical patches, MLSimp and S3 gridify trajectories, and TRMMA matches trajectories to the map. These operations block the gradients during the training of adversaries.

Could we borrow stones from other hills? There are some questions that are also open or solved problems for other unlearning fields, so we could learn from other fields to design better unlearning algorithms for trajectory data. The MLSimp algorithm utilizes mutual learning to optimize two models iteratively, which is similar to semantic communications [73] that optimize two models simultaneously. The map matching task is performed on the road network, unlearning the map matching model inherits the problems of unlearning on graphs [11, 87]. The unlearning effect of the User scenario is not as obvious as that of the Area scenario since the trajectories of the unlearning set of the User scenario are like randomly selected ones. This phenomenon is also demonstrated in literature on image classification tasks [24], i.e., the results of unlearning images in each class are not as impressive as unlearning the whole class. Therefore, it is possible to handle these problems by consulting unlearning algorithms designed for other fields.

9 Conclusion

We have presented the first study of machine unlearning on trajectory data. This is a crucial ingredient for providing updated services and protecting user privacy. Our investigation covered four key trajectory learning tasks, i.e., similarity learning, simplification, map matching, and recovery, each employing two SOTA models, across three real-world datasets and two unlearning scenarios. It studied different unlearning methods, including simple baselines, such as Finetune and NegGrad, teacher-student based methods, such as BadT and SCRUB, and information matrix based methods, such as SSD and SFRon, among others, adapted from the machine unlearning literature. Our investigation studied appropriate metrics including task and MIA metrics in order to substantiate and interpret results. Our results answer a large number of related key questions and also point out the characteristics of trajectory data and learning tasks. The work puts forth the basic skeleton, for instance, unlearning algorithms, performance metrics, downstream tasks, and unlearning scenarios, as well as related findings en route to a much-needed benchmark for unlearning on trajectory data.

Acknowledgments

This work is supported by Postgraduate Research & Practice Innovation Program of Jiangsu Province (Grant No. KYCX25_3466), by the Shenzhen University-Lingnan University Joint Research Programme (105023), by Project Funded by Priority Academic Program Development of Jiangsu Higher Education Institutions, and by the Institute of Applied Ethics at Soochow University, a Key Research Base of Philosophy and Social Sciences in Jiangsu Higher Education Institutions.

References

- [1] Donald J. Berndt and James Clifford. 1994. Using Dynamic Time Warping to Find Patterns in Time Series. In *KDD*. 359–370.
- [2] Lucas Bourtole, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. Machine unlearning. In *IEEE S&P*. 141–159.
- [3] Tianci Bu, Le Zhou, Wenchuan Yang, Jianhong Mou, Kang Yang, Suoyi Tan, Feng Yao, Jingyuan Wang, and Xin Lu. 2025. ProDiff: Prototype-Guided Diffusion for Minimal Information Trajectory Imputation. In *ICML*. 18 pages.
- [4] Weiyan Cao and Yunzha Li. 2017. DOTS: An online and near-optimal trajectory simplification algorithm. *Journal of Systems and Software* 126 (2017), 34–44.
- [5] Yinzi Cao and Junfeng Yang. 2015. Towards making systems forget with machine unlearning. In *IEEE S&P*. 463–480.
- [6] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. 2019. The secret sharer: evaluating and testing unintended memorization in neural networks. In *USENIX Security*. 267–284.
- [7] Yanchuan Chang, Jianzhong Qi, Yuxuan Liang, and Egemen Tanin. 2023. Contrastive Trajectory Similarity Learning with Dual-Feature Attention. In *ICDE*. 2933–2945.
- [8] Bing-Jyue Chen, Chiok Yew Ho, and De-Nian Yang. 2024. AFTER: Adaptive Friend Discovery for Temporal-Spatial and Social-Aware XR. In *ICDE*. 2639–2652.
- [9] Minxiao Chen, Haitao Yuan, Nan Jiang, Zhihan Zheng, Sai Wu, Ao Zhou, and Shangguang Wang. 2025. RLOMM: An Efficient and Robust Online Map Matching Framework with Reinforcement Learning. *Proc. ACM Manag. Data* 3, 3, Article 209 (2025), 26 pages.
- [10] Min Chen, Zhikun Zhang, Tianhao Wang, Michael Backes, Mathias Humbert, and Yang Zhang. 2021. When machine unlearning jeopardizes privacy. In *CCS*. 896–911.
- [11] Min Chen, Zhikun Zhang, Tianhao Wang, Michael Backes, Mathias Humbert, and Yang Zhang. 2022. Graph Unlearning. In *CCS*. 499–513.
- [12] Yile Chen, Gao Cong, and Cuauhtemoc Anda. 2023. TERI: An Effective Framework for Trajectory Recovery with Irregular Time Intervals. *Proc. VLDB Endow.* 17, 3 (2023), 414–426.
- [13] Yuqi Chen, Hanyuan Zhang, Weiwei Sun, and Baihua Zheng. 2023. RNTrajRec: Road Network Enhanced Trajectory Recovery with Spatial-Temporal Transformer. In *ICDE*. 829–842.
- [14] Ziwen Chen, Ke Li, Silin Zhou, Lisi Chen, and Shuo Shang. 2022. Towards robust trajectory similarity computation: Representation-based spatio-temporal similarity quantification. *WWW* 26, 4 (2022), 1271–1294.
- [15] Zhen Chen, Dalin Zhang, Shanshan Feng, Kaixuan Chen, Lisi Chen, Peng Han, and Shuo Shang. 2024. KGTS: Contrastive Trajectory Similarity Learning over Prompt Knowledge Graph Embedding. *AAAI* 38, 8 (2024), 8311–8319.
- [16] Dasol Choi and Dongbin Na. 2025. Distribution-level feature distancing for machine unlearning: towards a better trade-off between model utility and forgetting. In *AAAI*. 2536–2544.
- [17] Vikram S Chundawat, Ayush K Tarun, Murari Mandal, and Mohan Kankanalli. 2023. Can Bad Teaching Induce Forgetting? Unlearning in Deep Networks Using an Incompetent Teacher. *AAAI* 37, 6 (2023), 7210–7217.
- [18] Yizhou Dang, Yuting Liu, Enneng Yang, Guibing Guo, Linying Jiang, Jianzhe Zhao, and Xingwei Wang. 2025. Efficient and Adaptive Recommendation Unlearning: A Guided Filtering Framework to Erase Outdated Preferences. *ACM Trans. Inf. Syst.* 43, 2, Article 49 (2025), 25 pages.
- [19] DiDi. 2016. <https://outreach.didichuxing.com/en/>
- [20] European Parliament and Council of the European Union. 2016. *Regulation (EU) 2016/679 of the European Parliament and of the Council*. <https://data.europa.eu/eli/reg/2016/679/oj>
- [21] Ziquan Fang, Yuntao Du, Xinjun Zhu, Danlei Hu, Lu Chen, Yunjun Gao, and Christian S. Jensen. 2022. Spatio-Temporal Trajectory Similarity Learning in Road Networks. In *KDD*. 347–356.
- [22] Ziquan Fang, Changhao He, Lu Chen, Danlei Hu, Qichen Sun, Linsen Li, and Yunjun Gao. 2023. A Lightweight Framework for Fast Trajectory Simplification. In *ICDE*. 2386–2399.
- [23] Jie Feng, Yong Li, Kai Zhao, Zhao Xu, Tong Xia, Jinglin Zhang, and Depeng Jin. 2022. DeepMM: Deep Learning Based Map Matching With Data Augmentation. *TMC* 21, 7 (2022), 2372–2384.
- [24] Jack Foster, Stefan Schoepf, and Alexandra Brintrup. 2024. Fast Machine Unlearning without Retraining through Selective Synaptic Dampening. *AAAI* 38 (2024), 12043–12051.
- [25] Aditya Golatkar, Alessandro Achille, and Stefano Soatto. 2020. Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *CVPR*. 9304–9312.
- [26] Chuan Guo, Tom Goldstein, Awni Y. Hannun, and Laurens van der Maaten. 2020. Certified Data Removal from Machine Learning Models. In *ICML*, Vol. 119. 3832–3842.
- [27] Yang Guo, Zhiqi Wang, Jin Xue, and Zili Shao. 2024. A Spatio-Temporal Series Data Model with Efficient Indexing and Layout for Cloud-Based Trajectory Data Management. In *ICDE*. 1171–1184.
- [28] Chenxu Han, Sean Bin Yang, and Jilin Hu. 2026. DiffMM: Efficient Method for Accurate Noisy and Sparse Trajectory Map Matching via One Step Diffusion. In *AAAI*, Vol. 40. 14783–14791.
- [29] Chaowei He, Yuanjun Liu, Qingzhi Ma, Shenyuan Ren, Xizhao Luo, Lei Zhao, and An Liu. 2026. Forgetting by Pruning: Data Deletion in Join Cardinality Estimation. *AAAI* 40, 26 (2026), 21602–21609.
- [30] John Hersherberger and Jack Snoeyink. 1992. Speeding Up the Douglas-Peucker Line-Simplification Algorithm. In *SDH*. University of British Columbia, 134–143.
- [31] Mark He Huang, Lin Geng Foo, and Jun Liu. 2024. Learning to Unlearn for Robust Machine Unlearning. In *ECCV*. 202–219.
- [32] Zhehao Huang, Xinwen Cheng, Jinghao Zheng, Haoran Wang, Zhengbao He, Tao Li, and Xiaolin Huang. 2024. Unified Gradient-Based Machine Unlearning with Remain Geometry Enhancement. In *NIPS*, Vol. 37. 26377–26414.
- [33] Chih-Chieh Hung, Wen-Chih Peng, and Wang-Chien Lee. 2015. Clustering and aggregating clues of trajectories for mining trajectory patterns and routes. *VLDB J.* 24, 2 (2015), 169–192.
- [34] Jiawei Jiang, Dayan Pan, Houxing Ren, Xiaohan Jiang, Chao Li, and Jingyuan Wang. 2023. Self-supervised Trajectory Representation Learning with Temporal Regularities and Travel Semantics. In *ICDE*. IEEE.
- [35] Bingqing Ke, Jie Shao, Yi Zhang, Dongxiang Zhang, and Yang Yang. 2016. An Online Approach for Direction-Based Trajectory Compression with Error Bound Guarantee. In *APWeb*. 79–91.
- [36] Korbinian Koch and Marcus Soll. 2023. No matter how you slice it: Machine unlearning with sisa comes at the expense of minority classes. In *SaTML*. IEEE, 622–637.
- [37] Meghdad Kurmanji, Eleni Triantafillou, and Peter Triantafillou. 2024. Machine Unlearning in Learned Databases: An Experimental Analysis. *Proc. ACM Manag. Data* 2, 1, Article 49 (2024), 26 pages.
- [38] Meghdad Kurmanji, Peter Triantafillou, Jamie Hayes, and Eleni Triantafillou. 2023. Towards Unbounded Machine Unlearning. In *NIPS*, Vol. 36. 1957–1987.
- [39] Scott T. Leutenegger, Jeffrey M. Edgington, and Mario A. Lopez. 1997. STR: a simple and efficient algorithm for R-tree packing. In *ICDE*. 497–506.
- [40] Guanyao Li, Chih-Chieh Hung, Mengyun Liu, Linfei Pan, Wen-Chih Peng, and S.-H. Gary Chan. 2021. Spatial-Temporal Similarity for Trajectories with Location Noise and Sporadic Sampling. In *ICDE*. 1224–1235.
- [41] Jiaqi Li, Qianshan Wei, Chuanyi Zhang, Guilin Qi, Miaozen Du, Yongrui Chen, Sheng Bi, and Fan Liu. 2024. Single image unlearning: efficient machine unlearning in multimodal large language models. In *NIPS*. Article 1116, 40 pages.
- [42] Ruiyuan Li, Huajun He, Rubin Wang, Yuchuan Huang, Junwen Liu, Sijie Ruan, Tianfu He, Jie Bao, and Yu Zheng. 2020. JUST: JD Urban Spatio-Temporal Data Engine. In *ICDE*. 1558–1569.
- [43] Xiucheng Li, Kaiqi Zhao, Gao Cong, Christian S. Jensen, and Wei Wei. 2018. Deep Representation Learning for Trajectory Similarity Computation. In *ICDE*. 617–628.
- [44] Biwei Liang, Tengjiao Wang, Shun Li, Wei Chen, Hongyan Li, and Kai Lei. 2016. Online Learning for Accurate Real-Time Map Matching. In *PAKDD*. 67–78.
- [45] Shen Lin, Xiaoyu Zhang, Willy Susilo, Xiaofeng Chen, and Jun Liu. 2024. GDR-GMA: Machine Unlearning via Direction-Rectified and Magnitude-Adjusted Gradients. In *MM*. 9087–9095.
- [46] Yan Lin, Huaiyu Wan, Shengnan Guo, Jilin Hu, Christian S. Jensen, and Youfang Lin. 2024. Pre-Training General Trajectory Embeddings With Maximum Multi-Vent Entropy Coding. *TKDE* 36, 12 (2024), 9037–9050.
- [47] An Liu, Yifan Zhang, Xiangliang Zhang, Guanfeng Liu, Yanan Zhang, Zhixu Li, Lei Zhao, Qing Li, and Xiaofang Zhou. 2022. Representation Learning With Multi-Level Attention for Activity Trajectory Similarity Computation. *TKDE* 34, 5 (2022), 2387–2400.
- [48] Yu Liu, Qian Ge, Wei Luo, Qiang Huang, Lei Zou, Haixu Wang, Xin Li, and Chang Liu. 2024. GraphMM: Graph-Based Vehicular Map Matching by Leveraging Trajectory and Road Correlations. *TKDE* 36, 1 (2024), 184–198.
- [49] Yu Liu, Qian Ge, Wei Luo, Qiang Huang, Lei Zou, Haixu Wang, Xin Li, and Chang Liu. 2024. GraphMM: Graph-Based Vehicular Map Matching by Leveraging Trajectory and Road Correlations. *TKDE* 36, 1 (2024), 184–198.
- [50] Yuanjun Liu, An Liu, Guanfeng Liu, Zhixu Li, and Lei Zhao. 2023. Towards Effective Trajectory Similarity Measure in Linear Time. In *DASFAA*. 283–299.
- [51] Yuanjun Liu, Guanfeng Liu, Qingzhi Ma, Zhixu Li, Lei Zhao, and An Liu. 2025. GPE: Global Position Embedding for Trajectory Similarity Computation. In *KDD*. 1927–1938.
- [52] Ziqiao Liu, Hao Miao, Yan Zhao, Chenxi Liu, Kai Zheng, and Huan Li. 2024. LightTR: A Lightweight Framework for Federated Trajectory Recovery. In *ICDE*. 4422–4434.
- [53] Cheng Long, Raymond Chi-Wing Wong, and H. V. Jagadish. 2014. Trajectory simplification: on minimizing the direction-based error. *Proc. VLDB Endow.* 8, 1 (Sept. 2014), 49–60.
- [54] Yin Lou, Chengyang Zhang, Yu Zheng, Xing Xie, Wei Wang, and Yan Huang. 2009. Map-matching for low-sampling-rate GPS trajectories. In *GIS*. 352–361.
- [55] Zhipeng Ma, Zheyuan Tu, Xinhai Chen, Yan Zhang, Deguo Xia, Guyue Zhou, Yilun Chen, Yu Zheng, and Jiangtao Gong. 2024. More Than Routing: Joint GPS and Route Modeling for Refine Trajectory Representation Learning. In *WWW*. 3064–3075.

- [56] P. F. Marteau and G. M  nier. 2009. Speeding up simplification of polygonal curves using nested approximations. *Pattern Analysis and Applications* 12, 4 (2009), 367–375.
- [57] Nirvana Meratnia and Rolf A. de By. 2004. Spatiotemporal Compression Techniques for Moving Point Objects. In *EDBT*. 765–782.
- [58] Reham Mohamed, Heba Aly, and Moustafa Youssef. 2017. Accurate Real-time Map Matching for Challenging Environments. *TITS* 18, 4 (2017), 847–857.
- [59] Luis Moreira-Matias, Jo  o Gama, Michel Ferreira, Jo  o Mendes-Moreira, and Luis Damas. 2013. Predicting Taxi-Passenger Demand Using Streaming Data. *TITS* 14, 3 (2013), 1393–1402.
- [60] Jonathan Muckell, Paul W. Olsen Jr., Jeong-Hyon Hwang, Catherine T. Lawson, and S. S. Ravi. 2014. Compression of trajectory data: a comprehensive evaluation and new approach. *Geoinformatica* 18, 3 (2014), 435–460.
- [61] Paul Newson and John Krumm. 2009. Hidden Markov map matching through noise and sparseness. In *GIS*. 336–343.
- [62] Quoc Phong Nguyen, Bryan Kian Hsiang Low, and Patrick Jaillet. 2020. Variational bayesian unlearning. *NIPS* 33 (2020), 16025–16036.
- [63] M. Potamias, K. Patr  mpas, and T. Sellis. 2006. Sampling Trajectory Streams with Spatiotemporal Criteria. In *SSDBM*. 275–284.
- [64] Zeyuan Shang, Guoliang Li, and Zhifeng Bao. 2018. DITA: Distributed In-Memory Trajectory Analytics. In *SIGMOD*. 725–740.
- [65] Zhihao Shen, Kang Yang, Xi Zhao, Jianhua Zou, Wan Du, and Junjie Wu. 2024. DMM: A Deep Reinforcement Learning Based Map Matching Framework for Cellular Data. *TKDE* 36, 10 (2024), 5120–5137.
- [66] Weijie Shi, Jiajie Xu, Junhua Fang, Pingfu Chao, An Liu, and Xiaofang Zhou. 2023. LHMM: A Learning Enhanced HMM Model for Cellular Trajectory Map Matching. In *ICDE*. 2429–2442.
- [67] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership inference attacks against machine learning models. In *IEEE S&P*. 3–18.
- [68] Jianing Si, Haitao Yuan, Nan Jiang, Minxiao Chen, Xiao Ma, and Shangguang Wang. 2025. Towards Robust Trajectory Embedding for Similarity Computation: When Triangle Inequality Violations in Distance Metrics Matter. In *ICDE*. 1–14.
- [69] Yumeng Song, Yu Gu, Tianyi Li, Yushuai Li, Christian S. Jensen, and Ge Yu. 2024. Quantifying Point Contributions: A Lightweight Framework for Efficient and Effective Query-Driven Trajectory Simplification. *Proc. VLDB Endow.* 18, 2 (2024), 453–465.
- [70] Ayush K. Tarun, Vikram S. Chundawat, Murari Mandal, and Mohan Kankanalli. 2024. Fast Yet Effective Machine Unlearning. *TNNLS* 35, 9 (2024), 13046–13055.
- [71] Wei Tian, Jieming Shi, and Man Lung Yiu. 2025. Efficient Methods for Accurate Sparse Trajectory Recovery and Map Matching. In *ICDE*. 363–375.
- [72] Cheng-Long Wang, Qi Li, Zihang Xiang, Yinzhi Cao, and Di Wang. 2025. Towards lifecycle unlearning commitment management: measuring sample-level unlearning completeness. In *USENIX Security*. Article 333, 20 pages.
- [73] Weiqi Wang, Zhiyi Tian, Chenhan Zhang, and Shui Yu. 2025. SCU: An Efficient Machine Unlearning Scheme for Deep Learning Enabled Semantic Communications. *TIFS* 20 (2025), 547–558.
- [74] Weiqi Wang, Chenhan Zhang, Zhiyi Tian, and Shui Yu. 2025. FedU: Federated Unlearning via User-Side Influence Approximation Forgetting. *TDSC* 22, 3 (2025), 2550–2562.
- [75] Zheng Wang, Cheng Long, and Gao Cong. 2021. Trajectory Simplification with Reinforcement Learning. In *ICDE*. 684–695.
- [76] Zheng Wang, Cheng Long, Gao Cong, and Christian S. Jensen. 2024. Collectively Simplifying Trajectories in a Database: A Query Accuracy Driven Approach. In *ICDE*. 4383–4395.
- [77] Zheng Wang, Cheng Long, Gao Cong, and Qianru Zhang. 2021. Error-Bounded Online Trajectory Simplification with Multi-Agent Reinforcement Learning. In *KDD*. 1758–1768.
- [78] Dongen Wu, Ziquan Fang, Qichen Sun, Lu Chen, Haiyang Hu, Fei Wang, and Yunjun Gao. 2024. TrajRecovery: An Efficient Vehicle Trajectory Recovery Framework based on Urban-Scale Traffic Camera Records. In *KDD*. 5979–5990.
- [79] Leijie Wu, Song Guo, Junxiao Wang, Zicong Hong, Jie Zhang, and Yaohong Ding. 2022. Federated Unlearning: Guarantee the Right of Clients to Forget. *IEEE Network* 36, 5 (2022), 129–135.
- [80] Peilun Yang, Hanchen Wang, Defu Lian, Ying Zhang, Lu Qin, and Wenjie Zhang. 2022. TMN: Trajectory Matching Networks for Predicting Similarity. In *ICDE*. 1700–1713.
- [81] Di Yao, Gao Cong, Chao Zhang, and Jingping Bi. 2019. Computing Trajectory Similarity in Linear Time: A Generic Seed-Guided Neural Metric Learning Approach. In *ICDE*. 1358–1369.
- [82] Di Yao, Haonan Hu, Lun Du, Gao Cong, Shi Han, and Jingping Bi. 2022. TrajGAT: A Graph-based Long-term Dependency Modeling Approach for Trajectory Similarity Computation. In *KDD*. 2275–2285.
- [83] Jin Yao, Eli Chien, Minxin Du, Xinyao Niu, Tianhao Wang, Zezhou Cheng, and Xiang Yue. 2024. Machine Unlearning of Pre-trained Large Language Models. In *ACL*. 8403–8419.
- [84] Shanshan Ye and Jie Lu. 2024. Sequence Unlearning for Sequential Recommender Systems. In *Adv. Artif. Intell.* 403–415.
- [85] Shanshan Ye, Jie Lu, and Guangquan Zhang. 2025. Towards Safe Machine Unlearning: A Paradigm that Mitigates Performance Degradation. In *WWW*. 4635–4652.
- [86] Fudan Yu, Wenxuan Ao, Huan Yan, Guozhen Zhang, Wei Wu, and Yong Li. 2022. Spatio-Temporal Vehicle Trajectory Recovery on Road Network Based on Traffic Camera Video Data. In *KDD*. 4413–4421.
- [87] Chenhan Zhang, Weiqi Wang, Zhiyi Tian, and Shui Yu. 2024. Forgetting and Remembering Are Both You Need: Balanced Graph Structure Unlearning. *TIFS* 19 (2024), 6751–6763.
- [88] Yupu Zhang, Liwei Deng, Yan Zhao, Jin Chen, Jiandong Xie, and Kai Zheng. 2023. SimiDTR: Deep Trajectory Recovery with Enhanced Trajectory Similarity. In *DASFAA*. 431–447.
- [89] Silin Zhou, Yao Chen, Shuo Shang, Lisi Chen, Bingsheng He, and Ryosuke Shibasaki. 2025. Blurred Encoding for Trajectory Representation Learning. In *KDD*. 4132–4143.
- [90] Silin Zhou, Peng Han, Di Yao, Lisi Chen, and Xiangliang Zhang. 2023. Spatial-temporal fusion graph framework for trajectory similarity computation. *WWW* 26, 4 (2023), 1501–1523.
- [91] Silin Zhou, Jing Li, Hao Wang, Shuo Shang, and Peng Han. 2023. GRLSTM: trajectory similarity computation with graph-based residual LSTM. In *AAAI*, Vol. 37. 4972–4980.
- [92] Silin Zhou, Shuo Shang, Lisi Chen, Peng Han, and Christian S. Jensen. 2025. Grid and Road Expressions Are Complementary for Trajectory Representation Learning. In *KDD*. 2135–2146.
- [93] Zhiwei Zuo, Zhuo Tang, Kenli Li, and Anwitaman Datta. 2025. Machine Unlearning Through Fine-Grained Model Parameters Perturbation. *TKDE* 37, 4 (2025), 1975–1988.